

Allowance Audit



July 3, 2026

This document has been prepared solely for the named client and it may not be relied upon by any third parties. The content in this document is provided for informational purposes only and does not constitute professional advice of any kind. Security assessments are limited by factors such as scope, time, and techniques used and therefore this document does not constitute a certification, guarantee, or warranty regarding the proper functioning of any system or the absence of security vulnerabilities.

Copyright © 2026 Zeppelin Group Ltd.

Table of Contents

Table of Contents	2
Summary	3
Scope	4
System Overview	5
Security Model and Trust Assumptions	6
Notes & Additional Information	9
N-01 Pool-Short Withdrawal Failures Misattributed to Settled-Balance Checks	9
N-02 EObjectFundsWithdrawNotEnabled Mischaracterized as a Non-Matchable Execution Status	10
N-03 Incomplete Documentation	11
N-04 Divergences From Library-Wide Conventions	11
N-05 Non-Coin Objects Transferred to a Vault Are Permanently Unrecoverable	12
Conclusion	14
Appendix	15
Issue Classification	15

Summary

Type	Library	Total Issues	5 (5 resolved)
Timeline	From 2026-06-23 To 2026-06-25	Critical Severity Issues	0 (0 resolved)
Languages	Move	High Severity Issues	0 (0 resolved)
		Medium Severity Issues	0 (0 resolved)
		Low Severity Issues	0 (0 resolved)
		Notes & Additional Information	5 (5 resolved)
		Client Reported Issues	0 (0 resolved)

Scope

OpenZeppelin performed an audit of the [OpenZeppelin/contracts-sui](#) repository at commit [95dca41](#).

In scope were the following files:

```
contracts
├── allowance
│   ├── sources
│   └── spend_vault.move
```

Update: *The fixes for the findings highlighted in this report have all been merged at commit [cde1812](#).*

System Overview

The `openzeppelin_allowance` package provides a capability-keyed, multi-coin spending allowance primitive for Sui. It lets an owner (a treasury, protocol, or wallet) fund a shared escrow and delegate bounded, optionally expiring, revocable authority to spend from it, so a delegate can draw funds on demand without the owner relinquishing custody and without signing each individual spend. The engagement covered a single module, `spend_vault`. The three modules under `examples/spend_vault` (`direct_delegation`, `defi_keeper`, and `example_coin`) are out of scope; they were reviewed only as references for the intended integration patterns and not as audited code, and each is explicitly marked an unaudited illustration that is not production ready.

The central object is a shared, untyped `Vault` that holds multiple coin types at once. The pooled funds are not stored as a struct field: each coin lives as an object-owned address balance at the vault's own object address, and the only authority that can move funds out of the pool is a mutable reference to the vault's identifier, which the module produces solely behind a capability gate. There is no signer-based path to the pool. Per-grant state lives in a ledger keyed by the pair of spender-capability id and coin type, never in the capability itself, so a single capability can accrue independent budgets across many coins, and owner-side budget changes mutate the ledger entry in place while leaving the capability object and any downstream embedding untouched.

Authority is divided between two transferable capability objects, and the end-to-end flow moves through five stages: creation and funding, granting, spending, management, and teardown.

- **OwnerCap:** the owner authority for exactly one vault, minted at creation and consumed at teardown. It gates capability issuance, budget configuration, fund withdrawal, and vault destruction.
- **SpenderCap:** the spend authority. It carries no budget of its own; the owner grants per-coin budgets against its id, and whoever presents it can draw any coin it is budgeted for, up to each budget's limit and before its expiry.

An allowance is a ceiling on the shared pool rather than a reservation, so live budgets may sum to more than the pool by design, and competing spenders are resolved by consensus sequencing (first sequenced, first served). Budget amounts and expiries use a reserved maximum value as a sentinel for unlimited budget and for no expiry, so off-chain tooling that

totals spending volume must exclude that sentinel from arithmetic. Because the pool lives as object-owned balances at the vault address rather than in a struct field, only `Coin<T>` can be received back from that address; the recovery and teardown implications of that, the dependency on a Sui protocol feature for object-owned address balances, and the module's mixed error model are described below.

Security Model and Trust Assumptions

Vault Owner (OwnerCap holder): The owner mints spender capabilities, creates, raises, lowers, suspends, and revokes per-coin budgets, withdraws funds at any time, and destroys the vault. Exactly one `OwnerCap` exists per vault, so holding it is the entirety of owner authority and transferring it constitutes owner rotation. Owner exit operations (partial withdrawal, full settled-pool withdrawal, and destruction) consult only the capability binding and the pool, never the budget ledger, so no spender or ledger state can block defunding or teardown. Depositors and spenders must therefore trust the owner not to drain the pool out from under a live allowance: because budgets are ceilings on a shared pool and not reservations, the owner can lower, suspend, withdraw, or tear down at will, and a within-budget spend has no guarantee of being backed by funds.

Budget setting and revocation take the spender capability id as a bare, unvalidated argument, while the owner gate checks only the `OwnerCap`. A mistyped or wrongly copied id silently targets a different budget, the create path gives no distinct error signal because it provisions a fresh budget shaped exactly like a successful update, and a whole-capability revocation against a wrong id is a silent no-op that leaves the intended capability live. Owners should confirm the target before granting or revoking, for example by deriving the id from a held capability and checking the emitted events or a membership read. Teardown also requires owner discipline: destruction deletes the vault and drains the budget ledger but does not drain the pool, so any coin not withdrawn beforehand strands permanently at the dead vault address, and no on-chain guard can prevent a premature destruction because the runtime coin types held by a vault cannot be enumerated on-chain. Safe teardown depends on enumerating the vault address's coin types off-chain, withdrawing each, and confirming the pool is empty across a settled commit before destroying.

Because the set of granted coin types is never pruned, the owner is responsible for not granting an unbounded number of distinct coin types on a single long-lived vault; doing so grows the inline set without bound and inflates the Vault object toward the per-object size cap.

Spender (SpenderCap holder): The spender presents the capability to draw any coin the owner budgeted it for, within the remaining budget and before expiry. The capability is a bearer instrument: the library never inspects who holds or presents it, so possession is authority. Because a single untyped capability keys budgets across many coins, its effective authority is the sum of every per-coin budget it holds. The owner and any custodian must therefore treat the capability as secret-equivalent, since a leaked or mis-sent capability hands its full cross-coin spend authority to whoever receives it, and there is no recipient binding in the module. The owner-side mitigations are small per-coin budgets, finite expiries, suspension by setting a budget to zero, and a single-call whole-capability revocation that bounds a leaked capability's exposure to zero.

Depositor (permissionless): Anyone may deposit into the pool, whether through the module's deposit functions or by sending funds directly to the vault address, and depositing confers no rights, claim, or refund path. One consequence is that a withdrawal used as an emergency freeze is reversible: any party can re-arm a live allowance by topping the pool back up, so the durable way to stop a capability is revocation or destruction rather than draining the pool. Only a vault whose owner is trusted should be funded.

Because the vault is a shared object, its object id is also a valid address, so any account may transfer an object carrying the `store` ability to a vault, after which the object is owned by the vault. Only `Coin<T>` can be retrieved from a vault address, through the permissionless squash path; the module exposes no type-correct way to receive any other object type back. An NFT, a `TreasuryCap`, or either capability sent to a vault address therefore becomes permanently unrecoverable, and once the vault is destroyed it is stranded at a dead address. The most damaging case is sending an `OwnerCap` to its own vault, which loses the cap and disables all owner control while existing spender capabilities can still draw the pool down. The loss is silent, irreversible, and reachable by accident, since the funding documentation describes sending assets to the vault address without signaling that only coins are recoverable. Funds and objects should be sent to a vault only by parties that understand this limitation.

Integrators custodying a capability: Spending is capability-gated and never sender-gated, so any code that causes the library to observe a reference to a capability exercises that capability's full authority. The library delegates the access decision to the integrator. A protocol that custodies a user's capability must sender-gate the entry point that borrows it, because an ungated public borrow is world-drainable, and it must validate the capability's vault binding

against the intended vault before accepting custody. These two responsibilities are the integration's security boundary, not optional hardening.

A within-budget, unexpired spend is not guaranteed to be backed by funds. When the live pool (the balance at execution time, after every deposit and withdrawal already applied earlier in the same consensus commit) is short, the draw fails with the native

`InsufficientFundsForWithdraw` execution status rather than a module abort code. That status is not a matchable Move error code, so integrator preflight must dry-run the withdraw path and inspect transaction effects rather than match an abort code. Distinct from it, a standard matchable Move abort is raised by the framework when the object-funds-withdraw protocol feature is disabled, in which case balance reads can still report a non-zero value that cannot actually be spent or withdrawn.

Fund egress also relates to the settled balance, the accumulator value as of the start of the current consensus commit, which the module's balance reads return for advisory purposes. The settled and live balances coincide only at the start of each commit and diverge as the pool moves within it, so a value moved earlier in the same commit can cause a derived withdrawal to over-ask and fail; such failures are retry-safe on the next commit. The pool-short failure is therefore governed by the live balance, not the settled balance.

Deployment network (object-funds-withdraw feature): Every operation that moves funds out of the pool (spending, partial withdrawal, and full settled-pool withdrawal) routes through the framework's object-funds withdrawal primitive and therefore depends on the Sui object-funds-withdraw protocol feature being enabled on the network where the vault is deployed. The module assumes a deployment on a protocol version where this feature is active; it is currently enabled on Sui mainnet. Where the feature is disabled, the dependency is asymmetric across operations: deposits, whether through the module or by sending funds directly to the vault address, continue to succeed, and the balance and allowance reads remain total and still report non-zero, spendable-looking quotes, yet every spend and withdrawal aborts with the framework's matchable feature-flag error. A vault on such a network can thus be funded and appear to hold a drawable balance while no value can actually be drawn from it. Integrators targeting any environment other than current mainnet (a fork, a test network, or a future protocol change) should treat the feature as a standing liveness precondition rather than an always-available primitive.

Notes & Additional Information

N-01 Pool-Short Withdrawal Failures Misattributed to Settled-Balance Checks

The `Vault` holds funds as object-owned address balances in the Sui funds accumulator, and its fund-egress functions (`spend`, `withdraw`, and `withdraw_all`) draw from this pool. When a draw exceeds the available funds, the framework raises the `InsufficientFundsForWithdraw` execution status at `redeem_funds`. Two distinct quantities describe the pool, and the distinction governs when this failure occurs. The settled balance is the accumulator value as of the beginning of the current consensus commit. It is the figure recorded on the shared `AccumulatorRoot` object and returned by `settled_funds_value`. The live balance is the value at the moment of execution. It reflects every deposit and withdrawal already applied earlier in the same consensus commit, including earlier commands in the same programmable transaction block. The two quantities coincide at the start of each commit. They diverge while the pool moves within a commit, because accumulator deltas are aggregated and written back to the `AccumulatorRoot` object only at commit settlement, not on each operation.

The module documentation describes the pool-short failure as a check against the settled balance, stating that the abort occurs when the object's settled balance is below the requested amount. This is inaccurate. The `redeem_funds` primitive checks the live balance at execution time. It does not consult the start-of-commit snapshot read by `settled_funds_value`. Whenever the pool moves within a commit, the documented predicate becomes incorrect in both directions. A draw can succeed while the settled balance is below the amount, because a same-commit deposit raised the live balance. A draw can fail while the settled balance covers it, because a same-commit withdrawal lowered the live balance.

The inaccurate description appears in the following places:

- The module-level documentation, in both its [overview](#) and the [errors-section commentary](#)
- The `spend` function, in its [documentation](#), [abort list](#), and [commit-phase comment](#)
- The `withdraw` function, in its [documentation](#) and [abort list](#)

- The `withdraw_all` function, in a [parenthetical](#) within its documentation
- The package documentation, in the [README](#)

The `withdraw_all` occurrence is additionally self-contradictory. The surrounding prose correctly attributes the abort to the live pool falling below the settled snapshot. However, the parenthetical restates the cause as the settled balance being below the amount, immediately before a worked example in which the requested amount equals the settled value. Under that example the stated condition can never hold, yet the abort still occurs, because the true cause is the live balance.

Consider replacing each reference to the settled balance in the pool-short failure descriptions with the live balance, and reserving the term settled for the descriptions that genuinely use it, namely the `spendable_now` and `balance_value` reads and the `withdraw_all` amount selection.

Update: Resolved in [pull request #424](#).

N-02 `EObjectFundsWithdrawNotEnabled` Mischaracterized as a Non-Matchable Execution Status

The abort lists of the `spend`, `withdraw`, and `withdraw_all` functions document the failure that occurs when the `enable_object_funds_withdraw` protocol feature is disabled.

All three lists tag `EObjectFundsWithdrawNotEnabled` as an execution status, and the `spend` list additionally labels it "not a matchable Move `#[error]` code." This is incorrect. In the Sui framework, `EObjectFundsWithdrawNotEnabled` is declared as an `#[error]` constant with code 3 in the `funds_accumulator` module and is raised by an ordinary `assert`. It therefore produces a standard, matchable Move abort, which is categorically different from the native `InsufficientFundsForWithdraw` execution status that genuinely cannot be matched by code. Conflating the two could lead an integrator to believe the feature-flag failure cannot be matched by abort code when in fact it can.

Consider describing `EObjectFundsWithdrawNotEnabled` as a Move abort raised by the framework when the object-funds-withdraw feature is disabled, and reserving the "execution status" and non-matchable language for `InsufficientFundsForWithdraw` alone.

Update: Resolved in [pull request #425](#).

N-03 Incomplete Documentation

Several documentation passages are accurate in the common case but omit an edge case or elide a step, and could mislead an integrator:

- The `expiry` view describes a present entry's value as a future timestamp. Expired entries are not pruned, so the stored value may already be in the past at read time. An integrator who assumes a present entry's expiry is always in the future would treat an expired-but-unremoved grant as still valid. The module relies on this same non-pruning elsewhere, where a future expiry revives an expired entry in place.
- The suspension guidance, in both the [module-level overview](#) and the [set_allowance](#) documentation, states that the next spend aborts with `EAllowanceExceeded`. This holds only for a positive amount. A zero-amount spend aborts with `EZeroAmount` first, per the documented check order. The `EAllowanceExceeded` error definition already states the correct qualifier, "for any positive amount."
- The [deposit_balance](#) documentation sketches the funding flow as "`redeem_funds` then `deposit_balance`," eliding the `withdraw_funds_from_object` call that produces the `Withdrawal` argument consumed by `redeem_funds`.

Consider refining these passages so that each explicitly covers the omitted case and the full sequence of required calls.

Update: Resolved in [pull request #426](#).

N-04 Divergences From Library-Wide Conventions

The module conforms to the style guide in most respects. The following points diverge from the conventions the style guide defines:

- The module-level narrative headings use `####`, the heading depth the [style guide](#) designates for the per-function `Parameters`, `Returns`, and `Aborts` subsections, which the module also uses. In the generated reference, the [module narrative](#) therefore renders at the same depth as a function's parameter block. Consider using a shallower heading depth for the module-level narrative so that it is distinct from the function subsections.
- The `revoke_all` and `renounce` functions iterate with hand-rolled index loops, while the style guide lists the [loop macros](#) as the idiomatic form. Consider converting both to

`do_ref!` ; the rewrite is behavior-preserving. The remaining loop, in `destroy` , drains a `LinkTable` and has no loop-macro equivalent, so it is correctly left as is.

Consider aligning these passages with the style guide's conventions. The adjustments are behavior-preserving and leave the module's public interface unchanged.

Update: Resolved in [pull request #427](#).

N-05 Non-Coin Objects Transferred to a Vault Are Permanently Unrecoverable

A `Vault` in the `spend_vault` module keeps its per-coin pool as object-owned balances at the vault's own object address rather than in a struct field. Because the vault is a shared object, its object id is also a valid address, so any account may transfer an object carrying the `store` ability to a vault, after which the object is owned by the vault and can be retrieved only by receiving it through the vault's `&mut UID` . The module's funding and recovery surface is coin-typed throughout: the `deposit` and `deposit_balance` functions accept a `Coin<T>` or `Balance<T>` , a [// NOTE in the funding section](#) describes a direct `Balance` top-up via `send_funds` , and the permissionless `squash` path recovers a stray `Coin<T>` . No object-escrow pattern is presented.

The `squash` function is the only path that receives an object through the vault's `&mut UID` , and it is statically typed to `Receiving<Coin<T>>` . No other function receives through that `&mut UID` , and `destroy` deletes the vault without first receiving any owned objects. There is therefore no type-correct way to recover any other object: an NFT, a `TreasuryCap` , a `SpenderCap` , or an `OwnerCap` transferred to a vault address becomes permanently unrecoverable by any party, and once the vault is destroyed it is stranded at a dead address. This is the general transfer-to-object footgun on Sui, which applies to any shared object that exposes no matching typed receive for the object sent, rather than a behavior specific to this module. Because funding and recovery are coin-typed and no object-escrow flow is documented, the realistic trigger is a mis-addressed transfer rather than a documented pattern followed in good faith. The loss is bounded to the mis-sent object, cannot be triggered against another party, and is silent and irreversible.

The `OwnerCap` variant is the most consequential, as it both loses the capability and disables all owner control of the vault, since withdrawal, freezing, and teardown all require it, while existing spender capabilities can still draw the pool down. There is, however, no legitimate workflow that sends a vault's own `OwnerCap` to its address.

Consider documenting the limitation explicitly, stating that only `Coin<T>` can be recovered from a vault address and that any other object sent there is permanently lost. This keeps anyone relying on the documented address-level funding path from assuming the address behaves as a general escrow.

Update: Resolved in [pull request #428](#).

Conclusion

The `spend_vault` module implements a capability-keyed, multi-coin spending allowance primitive for Sui, letting an owner fund a shared vault and delegate bounded, optionally expiring, and revocable spend authority to others without surrendering custody and without signing each individual spend. OpenZeppelin audited this single module, with the surrounding integration examples reviewed only as references for the intended usage patterns.

The module uses a capability-based access model that separates owner authority from bearer spend authority and keeps every per-grant budget in a ledger keyed by capability and coin type, so grants stay composable and revocable in place. Much of the module's security rests outside its own boundary: spending is capability-gated rather than sender-gated, so the safety of any custodial integration depends on the integrator sender-gating the borrowing entry point and validating the capability's vault binding, and safe teardown depends on owner discipline to drain the pool before destruction. The reliance on a Sui protocol feature for object-owned address balances, and the error model in which a pool-short spend surfaces as a native execution status rather than a matchable abort code, are further conditions integrators must account for. Because the pool lives as object-owned balances at the vault address and only coins can be received back, any other object sent to that address is permanently lost.

Over the course of the audit, five informational notes were identified and no critical, high, medium, or low severity issues were found. Three concern the accuracy and completeness of the in-code and package documentation: the pool-short withdrawal failure is described as a check against the settled balance when the live balance governs it, the `EObjectFundsWithdrawNotEnabled` failure is labeled a non-matchable execution status when it is an ordinary matchable Move abort, and several passages omit an edge case or elide a required call. The remaining two cover divergences from the library's documentation and loop conventions and the permanent, silent loss of non-`Coin` objects transferred to a vault address. Given the bearer-capability model and the degree to which security depends on correct integration, maintaining accurate integrator-facing documentation and thorough test coverage of the spend, revocation, and teardown paths remains advisable.

The OpenZeppelin development team for Sui is thanked for their cooperation and responsiveness throughout the engagement.

Appendix

Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High
- Medium
- Low
- Note/Information

Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood of being exploited or may cause unwanted side effects in the system.

These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

Low Severity

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

Notes & Additional Information Severity

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.